

Project Documentation: 110101 Sequence Detector Module

Digital Design Lab

Project Overview

This design implements a parameterized **110101 Sequence Detector** in Verilog HDL. The circuit monitors a serial input stream (**din**) and asserts a detection flag (**detected**) whenever the bit sequence 110101 is matched. The module supports both **overlapping** and **non-overlapping** modes configurable via a module parameter (**OVERLAPPING**).

Specifications & Functionality

- Parameterization (OVERLAPPING):** Set to 1 to allow overlapping sequence detection (allowing the final bit of a matched sequence to serve as the start of the next sequence), or 0 for non-overlapping operation.
- State Machine States:** Six distinct states (S0 through S5) tracking the progress of the bit pattern 1-1-0-1-0-1.
- Asynchronous Reset:** Active-low reset (**reset_n**) forces the state register back to state S0.

Port Summary

Port Name	Direction	Type	Width	Description
clk	Input	wire	1	System clock input
reset_n	Input	wire	1	Asynchronous active-low reset
din	Input	wire	1	Serial input bit stream
detected	Output	reg	1	Sequence match detection output

Table 1: Sequence Detector Port Interfaces

Verilog Source Code

Design Module (seq_detector_110101.v)

```
module seq_detector_110101 #(
    parameter OVERLAPPING = 1
)(
    input  wire  clk,
    input  wire  reset_n,
    input  wire  din,
    output reg   detected
);
```

```
localparam S0 = 3'b000,
           S1 = 3'b001,
           S2 = 3'b010,
           S3 = 3'b011,
           S4 = 3'b100,
           S5 = 3'b101;

reg [2:0] current_state, next_state;

// State Register with Asynchronous Active-Low Reset
always @(posedge clk or negedge reset_n) begin
    if (!reset_n)
        current_state <= S0;
    else
        current_state <= next_state;
end

// Next State & Output Combinational Logic
always @(*) begin
    next_state = S0;
    detected    = 1'b0;

    case (current_state)
        S0: begin
            if (din) next_state = S1;
            else      next_state = S0;
        end

        S1: begin
            if (din) next_state = S2;
            else      next_state = S0;
        end

        S2: begin
            if (!din) next_state = S3;
            else      next_state = S2;
        end

        S3: begin
            if (din) next_state = S4;
            else      next_state = S0;
        end

        S4: begin
            if (!din) next_state = S5;
            else      next_state = S2;
        end

        S5: begin
            if (din) begin
                detected = 1'b1;
                if (OVERLAPPING)
                    next_state = S1;
                else
                    next_state = S0;
            end else begin
                next_state = S0;
            end
        end
    end
end
```

```

        end

        default: next_state = S0;
    endcase
end
endmodule

```

Testbench Module (tb_seq_detector.v)

```

`timescale 1ns / 1ps

module tb_seq_detector;

    reg clk;
    reg reset_n;
    reg din;

    wire detected_overlapping;
    wire detected_non_overlapping;

    // Instantiate Overlapping Detector
    seq_detector_110101 #(OVERLAPPING(1)) uut_overlapping (
        .clk(clk),
        .reset_n(reset_n),
        .din(din),
        .detected(detected_overlapping)
    );

    // Instantiate Non-Overlapping Detector
    seq_detector_110101 #(OVERLAPPING(0)) uut_non_overlapping (
        .clk(clk),
        .reset_n(reset_n),
        .din(din),
        .detected(detected_non_overlapping)
    );

    // Clock Generation (10ns Period)
    always #5 clk = ~clk;

    reg [0:10] test_stream = 11'b11010110101;
    integer i;

    initial begin
        clk      = 0;
        reset_n  = 0;
        din      = 0;

        #12;
        reset_n = 1;

        for (i = 0; i < 11; i = i + 1) begin
            @(negedge clk);
            din = test_stream[i];

            @(posedge clk);

```

```

        #1;
        $display("Time = %0t | Input = %b | Overlapping Out = %b |
                Non-Overlapping Out = %b",
                $time, din, detected_overlapping,
                detected_non_overlapping);

    end

    #20;
    $finish;
end

endmodule

```

Simulation & Verification

Execution Command

To run batch simulation using Cadence Xcelium or ModelSim[cite: 31]:

```
xrun seq_detector_110101.v tb_seq_detector.v
```

Expected Simulation Output

The simulation log evaluates both configurations against a continuous input bit stream[cite: 30, 31]:

```

Time = 17 | Input = 1 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 27 | Input = 1 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 37 | Input = 0 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 47 | Input = 1 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 57 | Input = 0 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 67 | Input = 1 | Overlapping Out = 1 | Non-Overlapping Out = 1
Time = 77 | Input = 1 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 87 | Input = 0 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 97 | Input = 1 | Overlapping Out = 1 | Non-Overlapping Out = 0
Time = 107 | Input = 0 | Overlapping Out = 0 | Non-Overlapping Out = 0
Time = 117 | Input = 1 | Overlapping Out = 1 | Non-Overlapping Out = 1

```